

Building with AI — Prototyping Your First App

Student study guide

Created by Swarup Biswas

Your take-home companion to the session. The slides carry the key ideas; this guide carries the explanation behind each one — plus the reference material we deliberately kept off the slides so the session could stay hands-on. Read it after class to consolidate, or before your next build.

What you should be able to do after this session

- Explain what a tech stack is and name the four layers every app shares.
- Explain how AI platforms collapse the stack — and build a working web app from a single written instruction.
- Write a five-part brief that gets good work out of an AI builder.
- Scope a first app correctly: one job first — the most lovable one, if it's for others.
- Run the build loop (Brief → Build → Poke → Patch) and stay in control of what the AI produces.
- Explain what a prototype is and is not, and what changes on the road from prototype to real product.

The one idea to carry out of the room: *You already know how to get good work out of people you don't manage — you brief them. AI building is the same skill: write a clear brief, build one job first, and poke the result before you trust it.*

1. A tech stack is a team of four

A **tech stack** is the set of technologies that work together to make an app run. Every app you have ever used — a messaging app, your bank, the decision app you built in class — is not one program but a *team* of cooperating parts, and the team has the same four roles in every app on earth.

Restaurant	The layer	What it does	In the app you built
Dining room	Front end	What users see and touch	The criteria rows, dropdowns, the two bars
Kitchen	Back end	Logic, rules, calculations — out of sight	Weight × score, summed per option → the Verdict
Storage room	Database	Where data survives between visits	Your decision survived the page refresh
Suppliers	Services	Outside help: login, payments, AI	The “suggest a criterion” button calling Gemini

Trace one everyday action — ordering food — and all four layers fire: the front end shows the menu and takes your tap; the back end checks the restaurant is open and prices the order; the database

records it and remembers your address; a payment service charges the card and another notifies the driver. One tap, four layers, invisibly coordinated.

Why it matters: you do not need to build any of these layers. But *naming* them changes how you work — when an app “forgets” your data, you now know that is a database problem; when a button does nothing, that is a front-end problem. Section 8’s fix prompts put this vocabulary to work.

2. The stack AI Studio actually scaffolded (verified)

We didn’t take Google’s word for it — we asked the generated app to report its own stack, and checked it against Google’s documentation.

In the app you built:

- **Front end:** React 19 with TypeScript, styled with Tailwind CSS v4 (plus two fonts and an icon set the AI chose by itself).
- **Back end:** the weighted-scoring logic, written as typed TypeScript running *in your browser* — no server, because the brief excluded one.
- **Database:** the browser’s own localStorage — “requiring zero databases or accounts,” which is exactly what “no backend, no login” produced.
- **Services:** empty — also by the brief. Until Patch 2.
- Behind the scenes: a build tool (Vite) that packages everything — a decision you never had to make.

When an app needs more, Build mode scales the same layers up (from Google’s documentation): a Node.js server runtime for secure API calls; Firebase — database and login — provisioned automatically the moment an app needs persistent data or sign-in; and the Gemini API wired in **server-side**, so the key “is never exposed in the browser” — a security decision made for you, worth knowing was made.

And we watched that happen. When Patch 2 asked for a Gemini-powered button, AI Studio converted the whole app to full-stack — a real Express backend appeared, with a dedicated endpoint, specifically so the AI key could stay server-side. The stack grew a layer the moment the services cell was filled. The four layers are not a diagram; they are how the machine actually organises itself.

Read the brief’s fingerprints in the stack: “no chart libraries” → the bars are styled div elements; “no backend, no login” → localStorage. And where the brief was silent, the AI decided alone: React 19, Vite, the typography, the entire visual theme. That is Section 5’s principle, demonstrated in your own build.

Sources: the generated app’s own stack report (11 Jul 2026); Google AI Studio Build mode documentation.

3. Reference: fifteen technology names you will hear

Kept off the slides deliberately. You do not need to learn these — recognise them so they stop being noise. The five in bold are the ones you will meet most often. Inside Scaffold Studio (Appendix A) these names come alive: each appears inside a scaffold with a visible job.

Technology	Layer	What it is, in one line
TypeScript	front/back	Typed JavaScript; catches bugs early. The default for serious web work — and what your app is written in.
React	front end	The dominant library for building app screens; what AI builders generate.
Tailwind CSS	front end	The styling toolkit that makes apps look the way modern apps look.
Node.js	back end	The most-deployed way to run JavaScript on a server.
PostgreSQL	database	The relational database that won. Usually met through Supabase.
Next.js	front+back	A React framework adding pages, routing, and server code.
Python	back end	The other default language — dominant in data and AI work.
Vercel	services	One-click hosting for web apps; where prototypes go live.
Supabase	db+services	Postgres plus login plus storage, packaged for non-experts.
shadcn/ui	front end	Ready-made, good-looking interface components.
Firebase	db+services	Google's app backend; what AI Studio auto-provisions.
Astro	front end	A framework optimised for content sites.
Vue	front end	React's main alternative; simpler mental model.
Bun	back end	A newer, faster alternative to Node.js.
Svelte	front end	A leaner React alternative.

4. Reference: the platforms — and the collapse

In 2026 the meaningful unit is not a stack but a platform that handles all four layers from a written description.

Platform	Best for	One-line profile
Google AI Studio	learning + free prototypes	Free, Gemini-powered, full-stack apps in minutes. What we used.
Lovable	polished products, non-coders	Market leader for non-coder building; used inside real companies. Limited free tier.
Claude Artifacts	fastest share	Prompt → app → one-button publish.
Bolt	in-browser dev, mobile	A full development environment in a browser tab.
v0 (Vercel)	interface design	Polished React interfaces; the designer's pick.
Cursor / Claude Code	real engineering	Developer tools where you read and steer the code — Rungs 2–3.

The point is the collapse, not the table. Assembling a stack took a team weeks in 2020; platforms now generate all four layers from a paragraph in minutes. The skill that stayed scarce is not assembling — it is **directing** the assembly. That is what this course teaches.

5. The brief: how to instruct an AI builder

You have briefed a renovation contractor, a vendor, an agency, a new hire. That skill — what I want, what it's for, what done looks like, what's out of scope — is exactly the skill that builds software now. A good brief has five parts:

#	Part	Skipping it costs you
1	What it is — one sentence	A meandering app that is several half-things
2	Who it's for — and the problem	A generic app nobody chooses
3	What it must do — numbered, specific	The AI decides for you — often wrongly
4	What it looks like — one real example	Placeholder emptiness you can't judge
5	What's out — stated exclusions	Logins and features you never asked for

The specificity principle — the deepest of the five: what you do not specify, the AI decides for itself. The industry's own documentation says so plainly — Lovable's prompting guide warns the tool “does not infer intent” and recommends founding prompts of 200–400 words, “longer than most beginners write.” Vague lines don't produce vague apps; they produce *confident apps that guessed*. Compare:

- *Vague*: “Users can manage their criteria.” → the AI just decided the control type, layout, defaults, delete behaviour, and empty state. Five guesses.
- *Specific*: “An input with an Add button; each criterion is a row with a delete button, a weight dropdown (1–5, default 3), and two score dropdowns.” → zero guesses.

Two habits that mark a professional-grade brief: **specify edge cases** (“if there are fewer than 3 criteria, show a message, not a broken chart”) and **specify empty states** (“every empty area shows a helpful placeholder line”). Ordinary users never think about the empty case; builders always do — and handled empty moments are the difference between an app that feels finished and one that feels abandoned.

Why a written brief beats building by chat: the AI plans against the complete picture, so the parts fit together; you are forced to think the idea through once (most “AI failures” are caught here, before generation); and the brief is reusable — same brief, new platform, second opinion in minutes.

Source: Lovable prompting documentation, 2025–2026.

6. Scoping: one job first — the lovable one, if it’s for others

The one-job rule: no feature #2 until feature #1 works, end to end. AI’s signature failure is abundance — ask for five features, get five at 80%, nothing finishes, and the app dies in a browser tab. One complete job always knows its next step. Professionals call the one-job version a **walking skeleton**: thin, but standing. Two reasons it wins with AI specifically: you learn in minutes whether the core idea works, and the AI extends whatever pattern it sees — a clean core gets cleanly extended; tangle gets more tangle.

The fork inside “who is it for”:

- Building for **yourself** → build the job you need. Convenience wins, ugly is fine, and this is the honest default for first apps.
- Building for **others** → “it works” is not the bar. Find the feature that would make someone **choose** your app — the *lovable feature* — and build it **first**, not last.

Naming the choose-it feature is a learnable skill:

App	Not the choose-it feature	The choose-it feature
A two-option decision helper	the criteria list	the two bars — the whole decision at a glance
An expense splitter for a group	the expense table	“who owes whom, settled in the fewest transfers”
A team leave planner	the calendar grid	“who’s out the same week as me”

The test: *what is the sentence someone says when recommending it?* Combined, the two rules give you a scoping sentence you can actually use: **build the one feature that would make your user choose this app — completely — and nothing else, until it works.**

7. The build loop: staying in control

Brief → Build → Poke → Patch. First output is not final output — that is the loop working, not you failing.

Poke — test before you trust. Click every control; type too-long answers; delete everything and look at what shows; refresh mid-task. The evidence for why this habit is non-negotiable (read directionally):

- **AI output looks more finished than it is.** Large-scale code analyses measure rising duplication and defect rates in AI-assisted code — while the code *reads* clean and confident. Polish is not evidence of correctness.
- **The feeling of speed is not evidence of speed.** In a 2025 randomised controlled trial (METR), experienced developers using AI tools *believed* they were ~20% faster and *measured* ~19% slower. Measure by poking, not by vibes. (Scope honesty: that trial studied experienced developers on mature codebases — keep the lesson, don’t over-quote the number.)

At prototype scale, poking IS the whole quality system. At Rung 3 the same instinct matures into professional code review and testing.

Patch — one change, aimed at the right layer. Batch three requests and you can’t tell what caused what. One change → poke → next change. And name the layer: a fix prompt that locates the symptom repairs faster than one that vents. Also know that AI sometimes “improves” things you didn’t ask about when applying a patch — a small price at this scale, and the exact reason professionals scope changes and review diffs at Rung 3.

Sources: GitClear code-quality research 2025; METR RCT 2025.

8. The patch library

The pattern: name the layer, describe the symptom, say what “fixed” looks like.

Symptom	Layer	Paste this
A button does nothing	front end	“The [X] button doesn’t do anything when I click it. Fix it.”
Broken on a phone	front end	“The layout is broken on my phone. Make it mobile-responsive.”
Bars/totals don’t update	back end	“The bars don’t update when I change a score. Make every score change update the bars immediately.”
Data vanishes on refresh	database	“My data disappears when I refresh. Save everything to localStorage and load it on startup.”
You made it worse	—	“Undo the last change — go back to the previous version.”

The undo prompt matters beyond repairs: knowing changes are reversible is what gives you permission to experiment.

The two class patches (both validated live):

Patch 1 — enhance a feature:

Make the winning option’s bar green and the other bar grey, and animate the bars when scores change.

Patch 2 — turn it into an AI app:

Add a Gemini call: a button labeled “Suggest a criterion I might have missed” that sends my decision and current criteria to the model and adds its one-line suggestion as a new criterion row.

What Patch 2 did in our validation run: AI Studio converted the app to full-stack (an Express backend appeared) so the Gemini key could stay server-side, added the endpoint and a styled button, and returned suggestions with an “AI Insight” explaining why each criterion matters. One prompt — and your tool became an AI app.

Annotation mode is the non-coder’s power tool for patches: click directly on any element of the app and describe the change in plain words — no need to know what anything is called.

9. Start from a scaffold, not a blank page

The bridge from writing briefs to real projects is the **scaffold**: a proven starting point for an app of a given shape — stack chosen, structure set, beginner traps avoided. Professionals rarely start from a blank page; they start from a template of the right shape and adapt.

The course’s catalog is **Scaffold Studio** — <https://scaffolding-applications.vercel.app/> — 24 scaffolds in five shapes, each teaching its stack and generating a paste-ready build prompt (with optional add-ons composed in). The full catalog is **Appendix A** of this guide.

The workflow it unlocks:

1. **Pick the shape** that matches your idea — AI app? dashboard? bot?
2. **Read the scaffold’s stack** — this is where Section 3’s names get jobs.
3. **Copy the ready-made prompt** — a brief an expert already wrote, all five parts present, stack decisions made.
4. **Adapt it with your own brief** — your what-it-is, your user, your worked example, layered on the scaffold’s foundation.

This is the honest answer to “how do professionals actually start an AI build?” — a shape decision, plus a proven starting prompt, plus their own specifics. Your next build practises this workflow end to end.

10. From prototype to product: the road ahead

	Rung 1 — Prototype (today)	Rung 2 — Scaffold (next)	Rung 3 — Practice (later)
The instruction	a brief (5 parts)	a PRD — brief + non-goals + testable requirements	spec → plan → tasks
Scope control	one job first	stated non-goals	tight scoping, do-not-touch zones
Quality control	poke it	the walking skeleton runs	review checklist + tests
Your 20%	taste	choice	review

When your app outgrows a brief, your brief grows into a spec. Nothing is abandoned; each of the five parts sharpens into its professional descendant — “what’s out” becomes non-goals, “what it must do” becomes testable requirements, “poke it” becomes review and tests.

The industry is walking the same road: **spec-driven development** — specifications structured enough that AI agents execute them directly (GitHub’s Spec-Kit is the emblematic project; “agentic engineering” is the emerging name for specifying before generating). This course is the entry ramp of that road.

Signals you’ve outgrown Rung 1: other people want to use your app (accounts, privacy); the data matters (backup, a real database); it needs to live somewhere reliable; changes keep breaking it. Each signal maps to a Rung 2–3 capability — and the sections below give you the starting points.

Making a PRD with AI — the three-prompt recipe. You don’t write a PRD from a blank page; you have the AI draft it and you make the decisions:

1. **Describe the problem** — not the app, the problem. Use this shape: *“I struggle with... I have tried... but...”*
2. **Discuss the scope** — *“help me decide the smallest lovable version of this.”* Push back on its suggestions; it’s a conversation, not an order.
3. **Ask for the draft** — *“write a PRD with the problem, the user, numbered requirements, and non-goals — under 500 words.”*

The division of labour is the familiar 80/20: the AI does the structure and the writing; you decide which features stay, what’s out, and what counts as success. The AI writes the draft, and you decide.

How the PRD guides the AI, step by step. A small app (3–5 features) can be pasted as one prompt — that’s what the class build did. A bigger app is built **one requirement at a time**: build requirement one, poke it, then requirement two. The PRD becomes the AI’s to-do list — and yours — and every step is the same Brief → Build → Poke → Patch loop. That is the whole secret of building big things with AI: a good document, taken one step at a time.

11. Anatomy of a good PRD — the industry-standard skeleton

The three-prompt recipe (§10) gets you a draft. This section is how you judge and improve that draft — the structure professional teams and AI-agent toolmakers converged on. Keep it beside you when

you read what the AI wrote.

The golden rule first: a PRD says WHAT and WHY — never HOW. The tech stack, the framework, the implementation — all deliberately excluded. They belong in a separate plan, decided later. This separation (the heart of GitHub’s Spec-Kit workflow) is what lets you change the how without rewriting the what — and it keeps the AI focused on your intent instead of its favourite technology.

The eight sections, and the job each one does:

#	Section	What goes in it	Why it’s there
1	Problem & why now	One paragraph: who has the problem, what it costs them, why solve it now	If this is weak, nothing below can save the app
2	Goals / Non-goals	Goals as <i>outcomes</i> , not features; non-goals stated bluntly	Non-goals are the #1 defence against AI scope-creep
3	Users & key scenarios	“As a [role], I want [capability], so that [benefit]”	Forces you to name a real user doing a real thing
4	Functional requirements	Numbered, testable statements: FR-1, FR-2...	Numbered so patches and reviews can point at them
5	Acceptance criteria	One “Given / When / Then” per requirement	The requirement’s proof — these become your pokes
6	Constraints & assumptions	Limits, rules, things that must NOT change	Unstated constraints get violated, politely
7	Out of scope / open questions	Unknowns, listed honestly	A gap gets filled by the AI’s imagination; a listed question gets flagged back to you
8	Implementation notes	<i>Left blank</i> — “deferred to the plan”	The WHAT/HOW wall, made visible

The three rules that make a PRD work with AI:

1. **Every requirement must be falsifiable.** If you can’t imagine a test for it, it’s not a requirement yet — it’s a wish. “Easy to use” is a wish. “A user can add a criterion in one click” is a requirement.
2. **State non-goals explicitly.** The AI ignores unstated boundaries — an unstated “don’t add login” gets a login screen.
3. **List open questions instead of leaving gaps.** This is the difference between the AI inventing an answer and the AI asking you.

A worked micro-example, from the app you built:

Level	Example
Wish (too vague)	“The verdict should feel responsive”
Requirement (FR-3)	“Every score or weight change immediately updates both bars and the verdict banner”
Acceptance criterion	“Given 3+ scored criteria, when any weight changes, then the bars re-scale and the banner re-evaluates — with no page refresh”

Notice: the acceptance criterion is literally a poke, written down in advance. A good PRD is a pre-written poking plan — which is why the build loop and the PRD are the same discipline at two different sizes.

The quality test for the whole document: could a stranger — human or AI — build the right app from this PRD *without asking you a single question*? Every question they’d need to ask is a hole in the document.

Sources: GitHub Spec-Kit (spec-driven development workflow); Anthropic’s guidance on agent-readable specifications; practitioner consensus on spec-writing in the AI era, 2024–2026.

12. Enrichment: the thinking tool inside the app you built

The technique your app implements — list what matters, weight each factor, score the options, multiply and sum — has a 250-year pedigree. In 1772, Joseph Priestley asked Benjamin Franklin for help with a career decision. Franklin’s reply described dividing a page into columns, writing every consideration down over several days, weighting them, and striking out items that balance — “*when each is thus considered separately and comparatively, and the whole lies before me, I think I can judge better, and am less likely to make a rash Step.*” He called it **moral or prudential algebra**; modern decision science calls its descendant a weighted decision matrix.

Franklin’s real insight explains why the app helps: hard decisions are hard “*chiefly because while we have them under consideration, all the reasons pro and con are not present to the mind at the same time.*” The app’s job is to hold everything in view at once.

When to trust the matrix, when to trust the gut: trust the matrix for decisions with several comparable factors — the value concentrates in arguing with your own weights, where people discover their real priorities. Override it *knowingly* when the verdict surprises you and the surprise won’t fade — a stubborn gut objection usually means a criterion is missing or a weight is dishonest. Fix the matrix, or trust the gut and say why. Both are legitimate; drifting silently between them is not. The app’s “Too close to call” banner is designed honesty: below a real margin, the maths has done its clarifying work and the casting vote is yours.

Source: Franklin to Priestley, 19 September 1772 (Founders Archives).

13. Your next build — this week

1. **Start from a real problem you have.** The documented pattern among non-developers who build successfully: the apps that survive are tools for a problem the builder actually has

— a lead scorer, a materials calculator, an ROI dashboard. Your recurring annoyance, put into software.

2. **Write your own five-part brief** for it — and notice which part is hardest. (It is usually part 5.)
 3. **Visit Scaffold Studio** (<https://scaffolding-applications.vercel.app/>) — find your idea’s shape, open its scaffold, copy its prompt. Appendix A previews the whole catalog.
 4. **If the idea is bigger than five features:** draft a PRD with the three-prompt recipe (§10), check it against the anatomy (§11), and build it one requirement at a time.
-

14. FAQ

Is what the AI builds real code? Yes — React/TypeScript projects, exportable as a ZIP or to GitHub, deployable to Google Cloud Run. “Prototype” describes the scope, not the materials.

Who owns what I build? The code is yours to export and keep. For commercial use, check the platform’s current terms of service.

Why did my app come out different from my neighbour’s? Generation varies run to run. What the brief specified is identical everywhere; what it left open, each generation decided differently. That IS the specificity lesson.

Could I just use a chatbot instead? For snippets, yes. An app builder plans, writes multiple files, assembles and hosts a running app — an agent that executes, not a chat that answers.

What does it cost? Prototyping in AI Studio: free. Costs begin at optional publishing at scale (the first two deploys are free as of mid-2026).

Is this “vibe coding”? The disciplined version of it: vibe coding meant accepting whatever the AI returns; this module adds a brief, a scope rule, and a verification habit — the same three things the industry’s own correction (spec-driven development) adds.

Will this replace developers? It replaces *waiting* for developers for small personal tools. Real products still require scoping, specifying, and reviewing — Rungs 2–3 skills. The realistic story is augmentation.

What if I want login / payments / many users? Deliberately out of Rung 1’s scope — services-layer capabilities with real responsibilities. The scaffold catalog (Appendix A) gives you starting points that support them properly.

15. The vocabulary

- **Tech stack** — the technologies that make an app run; four layers.
- **Front end / back end / database / services** — what you see / the logic / the memory / the outside help.
- **Full-stack** — spanning all four layers.
- **Agent** — AI that executes a task (plans, writes files, assembles), not just answers.
- **Brief** — a structured prompt with five parts; Rung 1’s instruction document.
- **Generation** — the plan → generate → assemble process; 1–3 minutes.
- **localStorage** — the browser’s built-in data store; a prototype’s database.

- **Walking skeleton** — the thinnest end-to-end version that works.
- **Lovable / choose-it feature** — what makes someone choose an app rather than tolerate it; built first when building for others.
- **Poke** — deliberate testing before trusting; matures into review.
- **Patch / fix prompt** — one targeted change, aimed at a named layer.
- **Annotation mode** — click an element, describe the change in plain words.
- **Scaffold** — a proven starting point with the stack chosen; your next build’s unit of work.
- **PRD / spec** — what a brief grows into as stakes rise (recipe §10, anatomy §11).
- **Spec-driven development** — specs structured enough for AI agents to execute; where this path leads.
- **The Builder’s Ladder** — Prototype → Scaffold → Practice. You are on Rung 1.

Read this list once a week after the session — explaining the four layers and the five brief parts in your own words is the readiness check.

Appendix A. The Scaffold Studio catalog

*All 24 scaffolds at <https://scaffolding-applications.vercel.app/> — browse by shape, open a scaffold to see its stack explained and copy its paste-ready prompt. **[ready]** = backed by a public reference repo today; **[soon]** = catalog entry with the prompt ready, reference repo in progress.*

Shape A — AI applications (the model is in the loop)

Scaffold	Stack	Use it for
A1 · FastAPI + Ollama + Postgres [ready]	local LLM, no cloud keys	Private/offline AI tools; data that can’t leave the machine (health, legal, HR)
A2 · FastAPI + Gemini + Postgres, Vercel/Render deploy [ready]	cloud LLM, split deploy	Public AI generators with a live URL; “form → LLM → saved result” products
A3 · Pydantic AI agent + FastAPI [soon]	tool-calling agent, typed outputs	AI that <i>does</i> things — queries data, calls APIs — not just answers
A4 · FastAPI + pgvector RAG [soon]	embeddings in Postgres	Document Q&A inside an app; semantic search; code-first RAG
A5 · n8n + Qdrant + Ollama (Docker) [ready]	low-code, self-hosted RAG	Chat-with-your-documents; compliance-sensitive knowledge bases
A6 · Python MCP server [soon]	Model Context Protocol	Extending AI assistants with your own tools; wrapping an internal API
A7 · FastAPI + Gemini vision upload [soon]	multimodal + file storage	“Upload a photo/PDF → AI analyses it” — receipts, forms, documents

Scaffold	Stack	Use it for
A8 · FastAPI voice (STT/TTS) [soon]	speech in and out	Voice Q&A, dictation tools, language practice

Shape B — Data apps & model demos

Scaffold	Stack	Use it for
B1 · Streamlit + Postgres [soon]	Python dashboards	KPI views, data exploration; the fastest “my analysis is now an app”
B2 · Gradio on Hugging Face [soon]	model demo UI	Shareable demos of a model you trained or fine-tuned

Shape C — Websites & dynamic web apps

Scaffold	Stack	Use it for
C1 · Single-file HTML [ready]	zero build	Landing pages live in an hour, hostable anywhere
C2 · Next.js + MDX content site [ready]	markdown-driven	Blogs, docs, course sites — “drop a .md file, push, it’s live”
C3 · FastAPI + HTMX [soon]	server-rendered, no React	Dynamic CRUD tools and admin panels for Python-first builders
C4 · React + Vite SPA [soon]	client-side app	Rich interactive frontends for any API
C5 · Vanilla PWA [soon]	installable, offline	Phone-installable tools without app stores
C6 · FastAPI WebSockets [soon]	real-time	Live chat, streaming AI replies, live dashboards

Shape D — No-UI apps (CLI, bots, automation)

Scaffold	Stack	Use it for
D1 · Python Typer CLI [soon]	installable command	Turning scripts into real command-line tools
D2 · Telegram bot [soon]	chat-channel app	Reminders, Q&A bots — an AI app in someone’s pocket, no frontend
D3 · Scheduled jobs (GitHub Actions) [soon]	serverless cron	“Email me when X changes” — monitors, scrapers, reports
D4 · Slack app (Bolt) [soon]	workplace bot	/ask commands, daily digests where professionals already work

Scaffold	Stack	Use it for
D5 · Playwright automation [soon]	browser robot	Automating sites with no API — logins, downloads, JS pages
D6 · Google Apps Script [soon]	zero infrastructure	Office automation on Sheets/Gmail — no hosting, no terminal

Shape E — Teaching & walkthrough repos

Scaffold	Stack	Use it for
E1 · Single-file FastAPI [ready]	one file, ~60 lines	The complete API mental model; mock APIs; teaching demos
E2 · Jupyter module repo [ready]	numbered lesson folders	Algorithm walkthroughs that build to a working artifact

Cross-cutting add-ons (toggle onto any scaffold)

1. **agents-config** — rules and skills for your AI coding assistant, adapted to the project.
2. **testing-ci** — pytest + GitHub Actions: “how do I know it still works?”
3. **llm-evals** — golden questions + assertions: “how do I know my AI app is *good*?” — the question that separates demos from products.

Four archetypes (SaaS starters with login/payments, browser extensions, native mobile, desktop) are deliberately pointed at maintained ecosystem starters instead of scaffolds — reach out if your idea lands there.

End of study guide. Companion materials: the slide deck, your app link, the build prompt with both patch prompts, and the session recording.